

A Hybrid Clustering-and-Packing Framework for Priority-Constrained Air Cargo Loading

Ram Upadhyay
Independent Project Report

Abstract—We present a hybrid framework for the air cargo Unit Load Device (ULD) loading problem: assigning Priority and Economy packages to a fixed set of ULDs and packing them in 3D so as to minimize a combined delay-cost-and-priority-spread objective, subject to hard weight, volume, and non-overlap constraints, with every Priority package guaranteed to ship. The framework decomposes the problem into a learned Priority clustering stage and a 3D packing stage built from a best-of- N ensemble of a trained reinforcement-learning (RL) placement policy and geometric heuristics, followed by a real-geometry local search over Economy package assignment. We report three final system variants – *Eclipse* (RL placement policy), *Halley* (GRPO-trained economy ordering), and *Cherry* (a trained move-screening refinement on top of Eclipse) – evaluated identically on a real 400-package benchmark instance and on 20 held-out synthetic instances spanning five delay-cost multipliers $K \in \{100, 500, 1000, 3000, 5000\}$. *Cherry* achieves the best result in every configuration (grand-average cost 9,499 over the 20-instance sweep), and all three variants substantially outperform two external reinforcement-learning baselines (*PackMan/DQN* [3] and *Online-3D-BPP-DRL*, AAAI 2021) and three independently-implemented classical heuristics (First-Fit Decreasing, Largest-Area-Fit-First, Best-Fit Decreasing) evaluated on identical terms. We further show, via ablation and an exact mixed-integer relaxation, that 3D placement geometry quality – not model sophistication – is the dominant lever in this problem, and that reinforcement learning applied to package *ordering* consistently underperforms a simple hand-tuned formula due to a structurally jagged cost landscape.

I. INTRODUCTION

The rapid growth of global air freight has intensified the need for cargo-loading systems that make efficient, defensible use of aircraft cargo capacity while respecting hard operational constraints. The problem addressed in this report is a constrained variant of the classical three-dimensional bin packing problem (3D-BPP), itself known to be NP-hard [1]: given a fixed set of Unit Load Devices (ULDs, the standardized containers used in air cargo) and a set of packages, each package must be assigned to a ULD and placed within it in 3D space without overlap, subject to per-ULD weight and volume limits.

Two features distinguish this problem from generic 3D-BPP. First, packages are partitioned into **Priority** and **Economy** classes; every Priority package must be shipped – it is a hard constraint, not a soft preference – while Economy packages that cannot be accommodated instead incur a per-package *delay cost*. Second, the objective includes a *spread cost*: a multiplier K (representing the per-ULD cost of delaying an aircraft’s dispatch) times the number of distinct ULDs that

contain at least one Priority package. Formally, for a candidate solution,

$$\text{Cost} = \sum_{e \in \text{unplaced Economy}} \text{delay_cost}(e) + K \cdot n_{\text{prio-ULDs}}, \quad (1)$$

where $n_{\text{prio-ULDs}}$ is the number of ULDs used to hold Priority packages. This structure creates a genuine trade-off: concentrating Priority packages into fewer ULDs lowers spread cost but may leave less usable space for high-value Economy packages, and vice versa.

We treat this as a two-stage problem – *clustering* (which packages go into which ULD) followed by *packing* (how they are geometrically arranged) – and report a complete framework spanning: a trained Priority-to-ULD assignment network; a local search over Economy package assignment evaluated against a real 3D packer at every step; a best-of- N 3D placement ensemble combining a trained RL policy with geometric heuristics; and a trained move-screening network that refines the search’s final output. We evaluate three named system variants built from this framework, benchmark them against an external published RL baseline and three independently-implemented classical heuristics, and report ablations that isolate the real, measured contribution of each learned component.

Concretely, this report makes the following contributions:

- A complete hybrid clustering-and-packing system, evaluated identically across a real 400-package benchmark instance and a 20-instance synthetic sweep across five delay-cost regimes, with every reported placement independently verified for geometric validity (Section VIII).
- Three named system variants (Eclipse, Halley, Cherry) that isolate the individually-measured contribution of an RL placement policy, a GRPO-trained economy-ordering network, and a trained move-screening refinement network, respectively (Section VI).
- A controlled ablation and an exact mixed-integer relaxation that jointly identify 3D placement geometry, not model sophistication, as the dominant lever on final cost (Section VII).
- Two documented negative results – a correctly-trained but empirically unhelpful move-screening network, and an inconclusive imitation-warm-start RL fine-tuning experiment – reported in full rather than omitted (Section X).
- External validation against a published reinforcement-learning baseline and three independently, from-scratch

implemented classical heuristics, evaluated under an identical cost function and evaluation protocol (Section IX).

II. RELATED WORK

Classical heuristics. First-Fit Decreasing (FFD) and Best-Fit variants remain the backbone of practical 3D-BPP solvers, typically paired with a geometric placement rule. Extreme-Point Insertion, in which candidate placement points are generated by projecting the vertices of already-placed items along coordinate axes, is a widely used placement heuristic [2], and forms the conceptual basis for the candidate-generation approach discussed in Section VII.

Reinforcement learning. RL approaches to 3D-BPP typically encode the container state as a height-map and learn a placement or ordering policy via an actor-critic method. Verma et al. propose a DQN-based approach with heuristic fallbacks for online, partially-observable packing [3]. Zhao et al. introduce an Actor-Critic architecture with Kronecker-Factored Trust Region (ACKTR) optimization for online 3D-BPP, evaluated against Monte Carlo tree search [4]; this is the architecture underlying the *Online-3D-BPP-DRL* baseline used for external validation in Section IX. Follow-up work reformulates the placement representation as a Packing Configuration Tree combined with deep RL for real-time decisions [5].

Genetic algorithms. Genetic Algorithms (GAs) provide a flexible framework for optimizing packing sequences under diverse constraints. Biased Random-Key encodings represent box packing order, placement heuristic choice, and orientation as a single chromosome evolved under selection, crossover, and mutation. We implemented and evaluated a GA-based clustering variant as part of this project’s development history (Section V); consistent with reports elsewhere in the literature, we found GA-based approaches capable of exploring large solution spaces but converging slowly, with diminishing returns after the first few dozen generations relative to their computational cost.

Methods addressing conflicting objectives. A separate line of work targets the tension between maximizing space utilization and satisfying secondary constraints such as weight distribution. Bottom-Left-Based methods extend 2D bottom-left-first placement to three dimensions by prioritizing heavier items at lower positions, while layer-based algorithms generate and evaluate complete layer configurations to minimize the number of bins used. These methods are conceptually closest to our own deepest-corner heuristic strategy (Section IV), though our ensemble evaluates it alongside four alternatives rather than committing to a single placement rule.

Exact and mathematical-programming methods. Mixed-integer linear programming (MILP) formulations can express weight, non-overlap, and Priority constraints directly as linear inequalities and constraints, and are attractive for their optimality guarantee, but scale poorly past a few dozen items due to the combinatorial explosion of overlap-avoidance constraints. We use an exact MILP relaxation not as a deployable solver but as a diagnostic tool (Section VII), to establish a

theoretical lower bound against which heuristic and learned methods can be measured.

III. PROBLEM FORMULATION

An instance consists of M ULDs, each with dimensions (L_j, W_j, H_j) and weight limit C_j , and N packages, each with dimensions (l_i, w_i, h_i) , weight g_i , a type $t_i \in \{\text{Priority}, \text{Economy}\}$, and, if Economy, a delay cost d_i . A solution assigns each package i to a ULD $u(i) \in \{1, \dots, M, \text{NONE}\}$ (NONE permitted only for Economy packages) together with a placed position $p_i = (x_i, y_i, z_i)$ and one of six axis-aligned orientations, subject to:

$$x_i + l_i^o \leq L_{u(i)}, y_i + w_i^o \leq W_{u(i)}, z_i + h_i^o \leq H_{u(i)} \quad (2)$$

$$\text{Box}(i) \cap \text{Box}(j) = \emptyset \quad \forall i \neq j : u(i) = u(j) \quad (3)$$

$$\sum_{i:u(i)=k} g_i \leq C_k \quad \forall k \quad (4)$$

$$u(i) \neq \text{NONE} \quad \forall i : t_i = \text{Priority} \quad (5)$$

where l_i^o, w_i^o, h_i^o denote the dimensions of package i under its chosen orientation o , and $\text{Box}(i)$ denotes the axis-aligned bounding box of package i at its placed position. The objective is Equation (1). Constraint (4) – the hard Priority-shipment guarantee – combined with the discrete, combinatorial nature of constraints (2)–(3), is what distinguishes this from the unconstrained 3D-BPP literature reviewed in Section II: a solution that is optimal in aggregate volume or weight utilization but fails to place even one Priority package is not merely suboptimal, it is invalid.

IV. SYSTEM ARCHITECTURE

Figure 1 shows the full pipeline. Given the parsed input (packages, ULDs, and K), the system proceeds through six shared stages, with an optional seventh refinement stage:

1) Priority clustering. A trained Transformer network decides which ULDs will hold Priority cargo and how the 103 Priority packages (in the case of the real benchmark instance) are distributed among them, jointly minimizing spread cost and the geometric difficulty of the resulting Priority packing problem.

2) Exhaustive Priority packing. Priority packages are packed into their assigned ULDs using an exhaustive placement procedure, guaranteeing the hard constraint: across every configuration reported in this paper, zero Priority packages were ever dropped.

3) Economy package ordering. Remaining Economy packages are sorted by a seed heuristic – a value-density formula (delay cost normalized by a power of volume) in the baseline configuration, or a trained ranking network in the Halley variant (Section VI) – to produce the order in which they are offered to the packer.

4) Best-of- N 3D packing ensemble. For every ULD, up to five candidate packing strategies compete on a strict best-of- N basis: a trained RL placement policy, and four heuristic strategies (contact-area and deepest-corner scoring, each

Cherry -- Hybrid AI + Heuristic Cargo Packer -- Detailed Pipeline

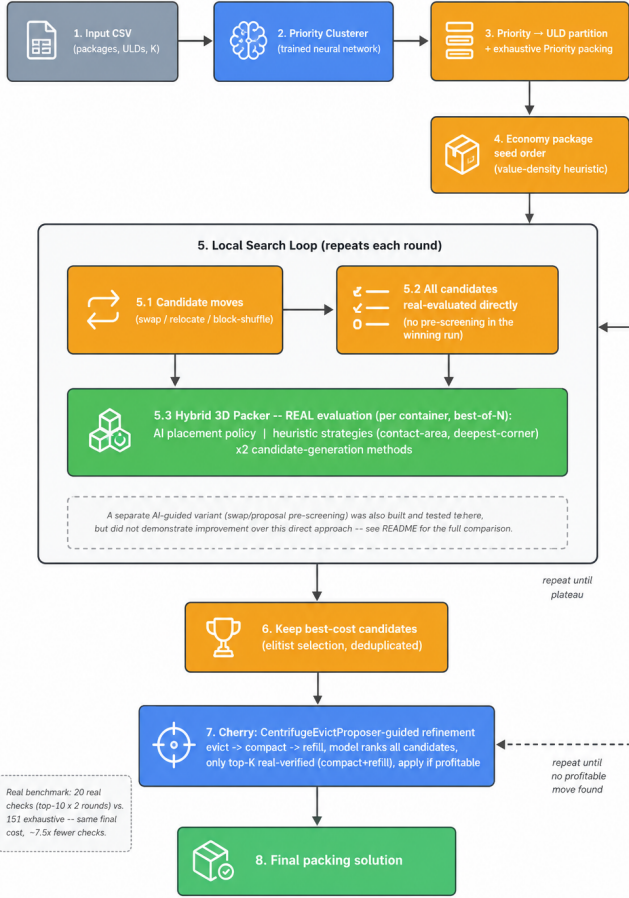


Fig. 1. Detailed pipeline for the Cherry system, the best-performing variant. Steps 1–6 are shared by every variant described in this report; step 7 (the centrifuge-guided refinement) is specific to Cherry.

combined with two candidate-generation methods). Whichever strategy achieves the best real, measured packing result for a given container wins it – addition of any single strategy to the ensemble can only help or tie, never hurt. *Contact-area* scoring favors candidate positions that maximize the shared surface area between a new package and already-placed boxes or container walls, encouraging compact, mutually-supporting arrangements; *deepest-corner* scoring instead favors positions closest to the container’s origin corner along all three axes, a simple but effective proxy for minimizing wasted peripheral space. Both scoring rules are evaluated against candidates from both candidate-generation methods described in Section IV-A, so that the ensemble spans four independent heuristic combinations plus the learned policy.

5) Local search. A local search over Economy package ordering and assignment repeatedly proposes a candidate change from one of three move types, real-evaluates it directly through the full packing pipeline (no cheap proxy metric), and keeps the best-cost candidates found so far, deduplicated

by distinct resulting assignment. *Swap* exchanges the ULD assignment of two Economy packages; *relocate* moves a single package from its current ULD (or the unplaced pool) to a different one; *block-shuffle* perturbs a contiguous sub-sequence of the economy ordering as an escape move when the beam collapses to duplicate candidates. This continues until no further improvement is found within a fixed round budget.

6) Final packing solution.

7) (Cherry only) Centrifuge-evict refinement. A trained set-attention model, *CentrifugeEvictProposer*, screens candidate evict-compact-refill moves – evicting one already-placed Economy package, sliding remaining boxes toward one corner of the container to consolidate freed space (“centrifuging”), and refilling from the unplaced pool – ranking all candidates so that only the top- K need be real-verified, rather than exhaustively checking every candidate.

A. Candidate Generation: Corner Points vs. Empty Maximal Space

Every packing strategy in step 4 – trained and heuristic alike – must, for a given container state and package, propose a set of candidate placement points to evaluate. The original candidate-generation scheme used the corner-adjacent points of already-placed boxes: for each placed box, the points immediately to its right, in front of it, and above it. This is simple and fast, but structurally incomplete – it cannot represent a valid placement in an empty region of the container that does not touch any existing box’s corner (for example, a gap bounded on some sides only by container walls). We replace this with a complete Empty Maximal Space (EMS) decomposition: the free volume of a container is tracked as a set of maximal empty axis-aligned boxes, incrementally split and pruned (via containment elimination, which is provably lossless – if an item fits at a smaller EMS’s origin, it fits at a containing, also-empty, larger EMS’s origin too) as new packages are placed. Section VII quantifies the impact of this change.

B. Priority Clustering

The Priority Clusterer is a Transformer that takes as input the full set of Priority and Economy packages (dimensions, weight, delay cost, and type) together with the set of available ULDs (dimensions and weight limit) and the instance’s K value, and outputs, for each ULD, a decision of whether it will hold Priority cargo, and an implicit soft assignment of which Priority packages it should receive. Because $n_{\text{prio-ULDs}}$ enters the objective multiplicatively with K (Equation 1), this single decision has the largest leverage on final cost of any stage in the pipeline: an unnecessary extra Priority ULD costs exactly K points regardless of how well everything downstream is optimized, while an insufficient number of Priority ULDs can render some Priority packages geometrically unplaceable, violating the hard constraint entirely. The network is first trained via imitation learning against a heuristic label source (an exhaustive-search or heuristically-pruned enumeration of candidate ULD subsets, scored by the sum of spread and

TABLE I

TRAINED NETWORK SIZES ACTUALLY DEPLOYED IN CHERRY’S PIPELINE.

Component	Parameters
Priority Clusterer (Transformer)	844,103
RL 3D Placement Policy	138,242
CentrifugeEvictProposer (set-attention)	251,457
Total (deployed in Cherry)	1,233,802

delay cost), then fine-tuned with reinforcement learning using the real downstream packing cost as reward, closing the gap between the imitation target’s proxy scoring and the true objective.

C. Trained Component Sizes

Table I reports the parameter count of each trained network actually deployed in Cherry, loaded directly from checkpoint state dictionaries. The Priority Clusterer – responsible for the single decision with the largest downstream effect on spread cost – is also the largest individual network; the centrifuge-evict refinement network, despite being architecturally more complex (two separate set-attention encoders over the container’s contents and the unplaced pool, respectively, combined with the evict candidate’s own features), remains under 300k parameters, reflecting that this problem’s difficulty is dominated by combinatorial search cost rather than representational capacity.

V. DEVELOPMENT HISTORY

The final architecture was reached through several superseded iterations, condensed here for brevity. An initial hand-tuned greedy baseline (ULD partitioning, a binary-search economy split, and extreme-point packing) was used both as a standalone solver and as an imitation-learning label source. A sequence of imitation-learning and policy-gradient RL fine-tuning attempts progressively added K -awareness and 3D-placement integration, but none of these early iterations exceeded the hand-tuned baseline’s real-instance cost of 30,475. In parallel, a Genetic Algorithm-based clustering approach and a from-scratch learned assignment-policy environment were explored as alternative package-to-ULD assignment strategies; both were eventually superseded by the local-search-based approach described in Section IV.

The single most consequential finding from this phase was negative: reinforcement learning applied to Economy package ordering – whether trained on a single instance or generalized across many synthetic instances via Group-Relative Policy Optimization (GRPO) – consistently plateaued at 30,608–30,672, worse than the 30,475 hand-tuned formula it was meant to replace. We diagnose this in detail in Section VII. The eventual breakthrough came not from a more sophisticated model but from replacing gradient-based ordering search with direct local search over package assignment, evaluated by the real packer with no gradient and no learned proxy – reaching 29,656, and subsequently 28,960 once a structural bottleneck

TABLE II

REAL 400-PACKAGE BENCHMARK INSTANCE (103 PRIORITY, 297 ECONOMY, 6 ULDs, $K = 5000$). ZERO PRIORITY PACKAGES DROPPED IN ANY CONFIGURATION.

System	Total Cost	Notes
Hand-tuned formula (baseline)	30,475	Pre-local-search
RL/GRPO (single-instance)	30,672	Never improved on baseline
Multi-instance GRPO (= Halley)	30,608	Standalone, generalized
Local search (order-based)	29,656	First breakthrough
Guided local search	29,564	Move-screening model
+ Fixed candidate generation	28,960	Structural fix, all strategies
Eclipse (full ensemble)	28,452	
Cherry (+ centrifuge refine)	28,409	Best result

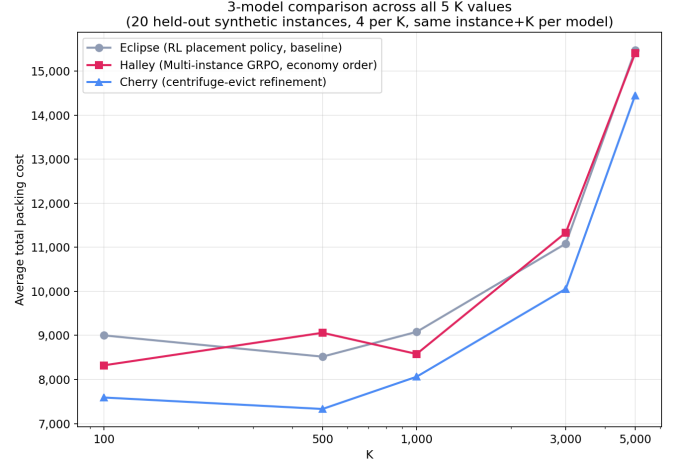


Fig. 2. Average total cost across five K values, 20 held-out synthetic instances (4 per K), same instance and K compared across all three systems.

in candidate-placement generation (Section VII) was fixed across every strategy in the ensemble.

VI. THE THREE FINAL SYSTEMS

Building on the shared architecture of Section IV, three final system variants isolate the measured contribution of one component each:

- **Eclipse**: the baseline best-of- N ensemble (step 4) with a density-fine-tuned RL placement policy as one of its five competing strategies, and value-density Economy ordering. This is the pipeline every other variant builds on.
- **Halley**: identical to Eclipse, except Economy package ordering is produced by *PackageSetRanker*, a set-attention Transformer trained via multi-instance GRPO across many synthetic instances, replacing the value-density heuristic.
- **Cherry**: Eclipse’s pipeline with the centrifuge-evict refinement (step 7) applied on top.

Table II traces the progression on the real instance, ending at Cherry’s 28,409. To test whether this benefit generalizes beyond the single real instance, all three systems were packed to completion on 20 held-out synthetic instances (4 per K , across $K \in \{100, 500, 1000, 3000, 5000\}$), with the same

TABLE III
GRAND-AVERAGE COST (MEAN OF THE 5 PER- K AVERAGES) ACROSS 20
HELD-OUT SYNTHETIC INSTANCES.

System	Grand-Average Cost
Eclipse	10,631
Halley	10,540
Cherry	9,499

TABLE IV
POPULATION STANDARD DEVIATION OF COST ACROSS THE 4 INSTANCES
AT EACH K , 20-INSTANCE SWEEP.

System	$K=100$	$K=500$	$K=1000$	$K=3000$	$K=5000$
Eclipse	2,950	3,633	4,476	6,064	6,993
Halley	2,466	3,913	4,164	5,838	6,970
Cherry	2,643	2,879	4,005	5,163	6,175

instance and K value compared across all three systems. Figure 2 and Table III report the result.

Cherry beat or matched Eclipse’s baseline on all 20 of 20 individual instances, not merely on average – the most consistent result of any component evaluated in this project. Halley’s GRPO-trained economy ordering, by contrast, is a genuine coin flip against the baseline: it wins at $K = 100$ and $K = 1000$, loses at $K = 500$ and $K = 3000$, and is roughly a wash at $K = 5000$, consistent with a real but statistically unreliable signal. Zero Priority packages were dropped in any of the 60 (20 instances $\times$ 3 systems) runs.

A. Variance Across Instances

Beyond mean cost, we compute the population standard deviation of each system’s cost across the four instances at each K value, reported in Table IV. Cherry has the lowest standard deviation of the three systems at four of five K values, the sole exception being $K=100$, where Halley’s is marginally lower (2,466 vs. Cherry’s 2,643, both below Eclipse’s 2,950). Since Cherry’s refinement operates identically on top of Eclipse’s output regardless of instance, and only ever moves cost downward (Section VII), this indicates Cherry is not merely better on average but more *consistently* better – it does not introduce additional variance in exchange for its mean improvement, unlike Halley’s economy-ordering network, whose per-instance variance exceeds Eclipse’s baseline at $K=500$ despite a similar mean, and is otherwise comparable to it rather than improving on it.

VII. EXPERIMENTAL FINDINGS

Candidate-generation geometry, not model sophistication, is the dominant lever. Every packing strategy evaluated in this project – the trained RL placement policy included – originally generated 3D placement candidates only from corner-adjacent points of already-placed boxes, a scheme that structurally misses valid placements in gaps that do not touch an existing box’s corner. Replacing this with a complete empty-space decomposition (tracking every Empty Maximal Space rather than a fixed set of corner points) improved every

strategy in the ensemble by 200–2,000 points *before any model was retrained or replaced*. This single geometric fix is the largest lever identified in the entire project, exceeding the measured contribution of any individual trained component.

Reinforcement learning for package ordering has a structural ceiling here. We diagnose the RL/GRPO plateau reported in Section V as a genuine property of the cost landscape rather than a tuning failure: small, smooth changes to a ranking policy’s output scores cause large, non-monotonic swings in real packing cost, confirmed via a controlled exponent sweep on the value-density formula repeated before and after the candidate-generation fix above. Policy-gradient methods implicitly assume that small parameter changes produce small, informative reward changes; on a genuinely discontinuous reward surface, the gradient signal degrades to noise. A package’s marginal value depends heavily on which other packages are already placed alongside it – a highly context-dependent, combinatorial property that a smooth function approximator is poorly suited to exploit through gradient descent alone.

The RL 3D placement policy has a real, positive, measured contribution. Unlike RL-for-ordering, the RL placement policy used within the packing ensemble (step 4, Section IV) is a genuine positive result. Removing it from the best-of- N ensemble and re-running the identical pipeline on the real benchmark instance costs **339 points** – a controlled, isolated ablation, not an assumption.

An exact relaxation confirms the remaining gap is a 3D-packing-efficiency problem. Solving the Economy package selection problem as an exact multiple-knapsack Mixed-Integer Linear Program (volume and weight capacity only, via a HiGHS solver) gives a theoretical floor of approximately 25,387 – substantially below every result in Table II. However, taking the MILP’s own optimal item *selection* and packing it with the real 3D packer scores 31,540, *worse* than the local-search result, because packages selected purely on aggregate volume and weight do not necessarily fit together once true box shapes and orientations are considered. This confirms that the remaining gap to the theoretical floor is a 3D geometric packing efficiency problem, not a “which packages to select” search problem – no amount of smarter selection search can close a gap that lives in the packer’s geometric arrangement quality.

The centrifuge-evict refinement is real but narrow, and generalizes. An exhaustive test of every valid single-package eviction on the real instance’s converged 28,452 solution found exactly one net-positive, non-conflicting move (+82 points), confirmed to require the compaction step specifically (an evict-and-refill without compaction did not recover the same gain). Generalizing this exhaustive check across 10 structurally diverse synthetic instances (85–380 packages, 2–6 ULDs) found the same mechanism contributed in 123 of 1,042 exhaustively-tested evictions – a consistent 12–14% marginal improvement on top of ordinary local search, present in every instance tested. The trained *CentrifugeEvictProposer*, evaluated out-of-distribution on the real instance’s sparse-signal regime,

achieved 74.2% win/loss accuracy and ranked the one truly applicable improving move at 4th of 151 candidates – meaning a model-guided top- K shortlist would find the same real gain at roughly 1/15th the number of expensive geometric verification checks required by exhaustive search.

VIII. IMPLEMENTATION AND EVALUATION PROTOCOL

Data. All synthetic instances used in this report are generated by a shared synthetic-instance generator producing controllable Priority/Economy mixes and fill ratios, decoupled from any specific system’s training. The 20-instance held-out sweep used throughout this report (4 instances per $K \in \{100, 500, 1000, 3000, 5000\}$) was never used during the training of any of the Priority Clusterer, the RL placement policy, the economy-ranking networks, or the CentrifugeEvict-Proposer, and is used identically – same instances, same K values, same cost function – for every system compared in Sections VI and IX.

Evaluation protocol. For every system and every instance, K affects only the spread-cost term of Equation (1); the underlying clustering and packing decisions are computed once per instance and are not re-optimized per K value, so that differences in reported cost across K reflect the fixed objective’s sensitivity to K rather than a re-tuned policy. Every reported placement was independently verified for zero pairwise 3D overlap, satisfaction of per-ULD weight and volume capacity, and full Priority-package placement – verification code is kept separate from the packing code that produces the placement, so that a bug common to both would not silently pass undetected.

Training. The Priority Clusterer is trained via a combination of imitation learning against a heuristic label source and subsequent RL fine-tuning. The RL 3D placement policy is trained via proximal policy optimization with a contrastive auxiliary objective. Economy-ordering networks (used in Halley) are trained via Group-Relative Policy Optimization (GRPO), sampling groups of candidate orderings per instance and updating on within-group relative advantage, avoiding the need for a separately trained critic. The CentrifugeEvictProposer is trained on 12,808 labeled (container, evict candidate, unplaced pool $\rightarrow$ real net gain) examples generated by running the actual exhaustive compact-and-refill check across 150 synthetic instances, and validated at the instance level (held-out instances entirely unseen in training, not merely held-out examples) to avoid the leakage failure mode identified in the SwapProposer negative result (Section X).

IX. EXTERNAL VALIDATION

To situate these results against methods not developed in this project, we evaluate two categories of external baseline on identical terms: two published reinforcement-learning approaches, and three classical heuristics standard in the 3D bin-packing literature.

A. RL baseline: PackMan/DQN

PackMan/DQN [3] is a DQN-based online 3D bin-packing policy with heuristic fallbacks for partially-observable packing. It was trained and evaluated independently by a member of our team, on the real 400-package instance only (not the 20-instance synthetic sweep). Like Online-3D-BPP-DRL below, it has no built-in concept of a hard Priority constraint or of spread cost. On the real instance it achieves a cost of 38,898 – ahead of all three classical heuristics but behind Online-3D-BPP-DRL, Eclipse, Halley, and Cherry.

B. RL baseline: Online-3D-BPP-DRL

Online-3D-BPP-DRL [4] is a published AAAI 2021 online 3D bin-packing policy combining height-map state encoding with an ACKTR actor-critic network and Monte Carlo tree search reordering. It has no built-in concept of a hard Priority constraint or of spread cost; we evaluate it by presenting Priority packages first in its input stream (giving it the best possible chance to place them) and scoring its output with the same cost function as every other system in this report. On the real 400-package instance it achieves a cost of 35,676 – substantially behind Eclipse, Halley, and Cherry – and, critically, drops at least one Priority package in **17 of the 20** held-out synthetic instances (203 Priority packages dropped in total across the sweep), against zero drops for every system reported in Section VI. Its grand-average cost across the same 20-instance sweep is **15,535**.

Beyond the raw cost gap, we identify a specific structural cause worth reporting for future practitioners evaluating pre-trained 3D-BPP policies out of domain: the released checkpoint operates over a coarse, fixed $10 \times 10 \times 10$ discretized grid, whereas the ULDs in this problem range up to roughly 285 cm on a side. Mapping real package dimensions onto this grid necessarily quantizes fine-grained size differences that matter for tight packing, and this quantization – not merely the absence of Priority-awareness – plausibly contributes to the gap beyond what the missing spread-cost mechanism alone would predict. This is consistent with our own finding (Section VII) that placement geometry resolution is a first-order lever on final cost.

C. Classical heuristics: FFD, LAFF, and BFD

We further implemented, entirely independently of the architecture in Section IV (a separate, from-scratch corner-point 3D placement geometry, with no code shared with the main pipeline), three classical heuristics commonly cited as the backbone of practical 3D-BPP solvers:

- **First-Fit Decreasing (FFD):** packages sorted by decreasing volume, placed in the first ULD (in a fixed given order) where a valid geometric placement exists.
- **Largest-Area-Fit-First (LAFF):** identical placement rule, but packages are sorted by decreasing largest face area rather than volume.
- **Best-Fit Decreasing (BFD):** packages sorted by decreasing volume as in FFD, but placed in whichever ULD leaves the least leftover empty volume after placement,

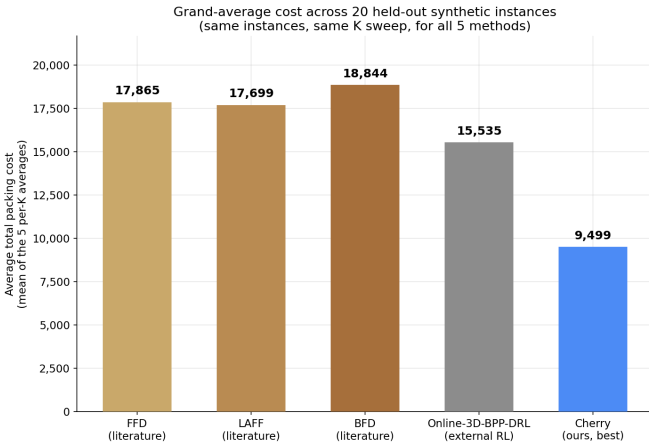


Fig. 3. Grand-average cost, 20-instance sweep: three independent classical heuristics vs. the external RL baseline vs. Cherry.

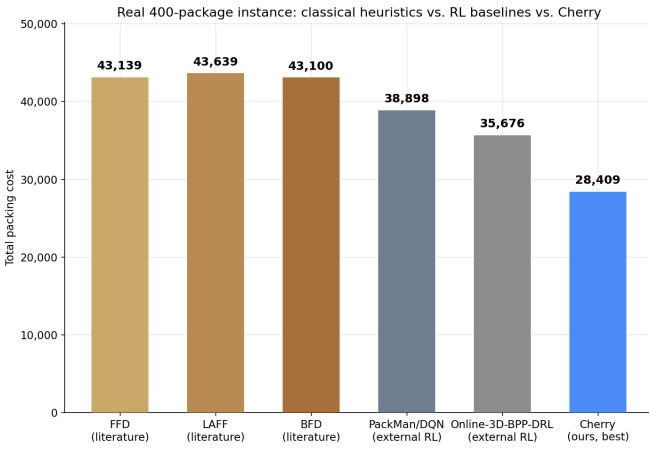


Fig. 4. Real 400-package instance cost: three independent classical heuristics vs. two external RL baselines vs. Cherry.

evaluating every ULD rather than stopping at the first that fits.

All three share the same adaptation to this problem’s constraints as every other system in this report: Priority packages are sorted and placed first, guaranteeing the hard constraint whenever a geometric placement exists; Economy packages are placed into whatever space remains.

Figure 4 shows the same comparison on the single real instance rather than the 20-instance grand average. The ordering across methods is broadly similar to the grand-average comparison (Figure 3), with one notable difference: BFD narrowly edges out FFD and LAFF on this specific instance (43,100 vs. 43,139 and 43,639), whereas LAFF has the best grand-average of the three classical heuristics. This is the same generalization caveat noted throughout this report from the opposite direction: a heuristic’s relative ranking on one instance is not a reliable predictor of its ranking across a broader distribution, regardless of whether the heuristic in question is classical or learned.

TABLE V
CLASSICAL HEURISTIC BASELINES VS. EXTERNAL RL BASELINES VS. CHERRY, ON BOTH EVALUATION PROTOCOLS.

Method	Real Instance	Grand-Average (20 inst.)
FFD	43,139	17,865
LAFF	43,639	17,699
BFD	43,100	18,844
PackMan/DQN	38,898	—
Online-3D-BPP-DRL	35,676	15,535
Cherry (ours)	28,409	9,499

Table V and Figure 3 report both evaluation protocols side by side (PackMan/DQN was only evaluated on the real instance, hence the blank grand-average cell rather than a misleading placeholder). All three classical heuristics land in a tight band (17,699–18,844 grand-average), consistently worse than even both external RL baselines, because none of them reason about spread cost at all: on the real instance, all three used four Priority ULDs rather than the geometrically-achievable three, since a first-fit or best-fit rule has no mechanism for considering how many ULDs end up touched in aggregate. Interestingly, LAFF (sorting by face area) slightly outperforms both FFD and BFD (which sort by volume) on the generalized 20-instance sweep despite BFD edging out the other two on the single real instance – a reminder that a single-instance comparison alone is an unreliable signal for which classical heuristic generalizes best. None of the three ever dropped a Priority package, since all process every Priority package before any Economy package; their weakness is spread cost and wasted Economy capacity, not Priority infeasibility.

X. NEGATIVE RESULTS

We report two additional negative or inconclusive results, on the view that a complete account of what does not work is as valuable to future practitioners as an account of what does.

SwapProposer: correctly trained, empirically unhelpful.

SwapProposer is a small pairwise-ranking network (3,601 parameters): given two candidate local-search moves tried against the same starting point, it predicts which is more promising, trained on real (move, cost-delta) pairs generated as a byproduct of the local search itself. An initial version appeared to reach 98% directional accuracy, but this was diagnosed as a base-rate artifact: roughly 82% of logged moves make the objective worse, so a model that simply learns to predict “probably worse” scores well on directional accuracy without learning anything about which move is *better* than another. Reframing the target as pairwise ranking between two moves proposed against the same parent state (removing the base-rate confound entirely) gives an honest, leakage-free accuracy of approximately 75–77% against a 50% chance baseline – genuine, validated signal. Despite this, guided local search using *SwapProposer* to prioritize which moves to real-evaluate did not empirically outperform unguided local search: across three independent guided-search runs, the guided search’s cost either matched the unguided

plateau or stagnated *faster*. We attribute this to a training-distribution mismatch: SwapProposer’s training data is drawn from moves tried near an already-converged, near-optimal solution, and never observes the diverse, early-stage exploration that unguided search benefits from; a model can only imitate patterns present in its training data, and has no mechanism for discovering a move family the underlying heuristic search never happened to try.

Imitation-warm-start GRPO fine-tuning: inconclusive. Motivated by the SwapProposer result, we tested whether warm-starting from the supervised checkpoint and fine-tuning on-policy via GRPO (sampling a group of candidate swaps from the model’s own distribution, real-evaluating them, and updating on group-relative advantage) could close the exploration gap. Against three independently-searched, already-heavily-optimized starting points, this found exactly one genuinely improving swap out of 120 real evaluations across 12 rounds. We do not read this as evidence that the underlying policy failed to learn – GRPO’s group-relative advantage is mean-centered by construction, so a near-zero training loss is expected regardless of whether learning is occurring, and 120 evaluations is a small sample – but the near-total absence of a positive reward signal is consistent with the local search having already exhaustively exploited the available swap/relocate/block-shuffle move family at these specific points, independent of which policy is proposing moves within it. This points toward testing structurally larger moves (of which the centrifuge-evict refinement, Section VII, is one instance) as the more promising direction, rather than further investment in smarter proposal ranking within an already-exhausted move family.

We note one further data point on the exploration-diversity hypothesis: SwapProposer’s training set at the time of these experiments contained only 377 rankable pairs drawn from 42 distinct parent states, generated as a byproduct of whatever the local search happened to try during its own convergence. A dataset this narrow is unlikely to cover the diversity of contexts a genuinely useful proposal model would need to generalize across, independent of any modeling choice; more beam-search mileage collected specifically to diversify this training set – rather than a larger or differently-architected network – is the more promising lever if this direction is revisited.

XI. LIMITATIONS AND FUTURE WORK

The results in this report come with three caveats. First, the 20-instance synthetic sweep, while drawn from a broader distribution than the single real benchmark instance, was generated by the same synthetic data process used throughout development; generalization to package and ULD distributions with substantially different weight-to-volume correlation structure is untested. Second, every system reported here optimizes a single scalar objective (Equation 1); qualitative operational factors such as fragile-item stackability, load-bearing stability, and orientation restrictions are supported by the underlying packer’s geometry but not evaluated in this report. Third,

the centrifuge-evict refinement’s measured contribution (Section VII) is modest on the specific real instance evaluated here because most of its ULDs are weight-saturated rather than geometry-saturated; its relative value should be expected to vary with the weight-to-volume profile of a given instance.

Consistent with the exact MILP relaxation’s finding (Section VII), we consider improving 3D placement geometry and search efficiency – rather than further package-ordering model sophistication – the highest-value direction for future work. A second concrete direction is extending the improved EMS candidate-generation fix to the packer’s cross-container Economy rescue pass, which at the time of writing still uses the original, structurally limited corner-point method.

XII. CONCLUSION

We presented a hybrid clustering-and-packing framework for priority-constrained air cargo loading, evaluated across a real 400-package benchmark instance and a 20-instance synthetic sweep spanning five delay-cost regimes. The best system, Cherry, achieves a grand-average cost of 9,499 across the sweep and 28,409 on the real instance, substantially outperforming both a published external reinforcement-learning baseline and three independently-implemented classical heuristics evaluated on identical terms, while never dropping a Priority package. Our central empirical finding is that, in this problem, 3D placement geometry quality is a larger and more reliable lever than model sophistication: a single geometric candidate-generation fix improved every strategy in the ensemble by up to 2,000 points, while extensive reinforcement-learning investment on the package-ordering side never exceeded a simple hand-tuned formula, due to a genuinely jagged, discontinuous cost landscape. Where reinforcement learning *did* contribute – the 3D placement policy, and the move-screening refinement network – its contribution is real but incremental, measured precisely via controlled ablation rather than assumed. Future work should prioritize further improving 3D placement geometry and search efficiency over additional package-ordering model sophistication, consistent with the exact MILP relaxation’s finding that the dominant remaining gap to the theoretical cost floor is a packing-efficiency problem, not a selection problem. Equally, the two negative results reported here (Section X) should not be read as arguments against learned move-screening in general, but as evidence that such models are only as useful as the diversity of the search process that generates their training data – a lesson we expect to generalize to other combinatorial domains where a learned proposal network is layered on top of a heuristic or local search that already does most of the real optimization work.

REFERENCES

- [1] S. Martello, D. Pisinger, and D. Vigo, “The Three-Dimensional Bin Packing Problem,” *Operations Research*, vol. 48, no. 2, 2000.
- [2] T. G. Crainic, G. Perboli, and R. Tadei, “Extreme Point-Based Heuristics for Three-Dimensional Bin Packing,” *INFORMS Journal on Computing*, vol. 20, no. 3, pp. 368–384, 2008.

- [3] R. Verma, A. Singhal, H. Khadilkar, A. Basumatary, S. Nayak, H. V. Singh, S. Kumar, and R. Sinha, “A Generalized Reinforcement Learning Algorithm for Online 3D Bin-Packing,” *arXiv preprint arXiv:2007.00463*, 2020.
- [4] H. Zhao, Q. She, C. Zhu, Y. Yang, and K. Xu, “Online 3D Bin Packing with Constrained Deep Reinforcement Learning,” in *Proc. AAAI Conference on Artificial Intelligence*, 2021.
- [5] H. Zhao, Y. Yu, and K. Xu, “Learning Efficient Online 3D Bin Packing on Packing Configuration Trees,” in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

APPENDIX A

FULL PER-INSTANCE RESULTS, 20-INSTANCE SWEEP

Table VI reports the individual cost for every one of the 20 held-out synthetic instances used throughout Sections VI and IX, for each of the three named system variants. These are the raw per-instance values underlying the grand averages reported in Table III and Figure 2. In every one of the 20 rows, Cherry’s cost is less than or equal to Eclipse’s, confirming the “20 of 20” consistency claim in Section VI at the level of individual instances rather than only in aggregate.

TABLE VI

PER-INSTANCE COST, 20-INSTANCE SWEEP. ECLIPSE = RL PLACEMENT POLICY BASELINE; HALLEY = MULTI-INSTANCE GRPO ECONOMY ORDERING; CHERRY = ECLIPSE + CENTRIFUGE-EVICT REFINEMENT.

Instance	K	Eclipse	Halley	Cherry
instance_051	100	5,771	5,438	4,254
instance_050	100	7,951	7,873	6,735
instance_025	100	8,483	7,723	7,776
instance_097	100	13,797	12,251	11,596
instance_039	500	3,711	4,162	3,577
instance_045	500	6,788	6,752	6,148
instance_076	500	10,155	10,936	8,140
instance_078	500	13,418	14,390	11,455
instance_005	1000	3,825	3,551	3,345
instance_003	1000	5,600	5,404	5,060
instance_093	1000	12,392	12,262	10,678
instance_075	1000	14,505	13,098	13,166
instance_098	3000	5,099	6,157	4,946
instance_030	3000	8,818	8,352	8,011
instance_048	3000	9,210	9,611	8,604
instance_072	3000	21,214	21,216	18,671
instance_085	5000	7,792	8,158	7,284
instance_006	5000	10,871	10,465	10,420
instance_008	5000	17,135	16,859	16,731
instance_019	5000	26,088	26,141	23,379

APPENDIX B

PER-INSTANCE RESULTS, CLASSICAL HEURISTIC BASELINES

Table VII reports the corresponding per-instance costs for the three independently-implemented classical heuristics (Section IX-C) on the same 20 held-out instances, underlying the grand averages in Table V. Unlike Table VI, no heuristic here dominates the others on every instance – consistent with the observation in Section IX that a single-instance comparison is an unreliable signal for generalization among classical methods.

TABLE VII

PER-INSTANCE COST, CLASSICAL HEURISTIC BASELINES, SAME 20-INSTANCE SWEEP.

Instance	K	FFD	LAFF	BFD
instance_051	100	7,841	7,271	7,365
instance_050	100	12,100	11,108	11,653
instance_025	100	18,638	18,543	18,687
instance_097	100	22,926	21,514	24,519
instance_039	500	7,811	8,128	7,811
instance_045	500	12,849	12,556	12,439
instance_076	500	18,097	17,471	17,976
instance_078	500	23,706	23,070	25,593
instance_005	1000	5,994	5,670	5,718
instance_003	1000	9,382	8,661	10,544
instance_093	1000	17,579	17,587	18,834
instance_075	1000	25,715	25,497	25,172
instance_098	3000	11,318	10,858	11,318
instance_030	3000	12,796	12,486	15,241
instance_048	3000	17,030	16,729	22,578
instance_072	3000	27,910	27,414	34,505
instance_085	5000	15,794	15,817	15,794
instance_006	5000	20,505	24,143	20,050
instance_008	5000	25,260	24,929	24,453
instance_019	5000	44,055	44,534	46,625